

```
{
  num_jobs_processing: 0,
  num_jobs_completed: 13,
  num_jobs_failed: 0,
  gb_in_process: 0,
  num_assemblies_completed: 150,
  num_assemblies_failed: 1,
  num_assemblies_processing: 12,
  num_batches_processing: 0,
  num_batches_completed: 1,
  num_batches_failed: 0,
  number_of_active_replications: 0,
  gb_to_be_replicated: 0,
  audit_online_file_count: 31600,
  gb_audited_online: 145.11,
  audit_nearline_file_count: 2128,
  oldest_audit_in_days: 54,
  gb_ingested: 9
}
```

MERRITT REPOSITORY AT A GLANCE

- 105 million files
- 640 TB primary copy
- 4.8 million objects
- 1.86 PB for 3 copies
- Strict, forward-delta object versioning
- Managed at the file level
- 14 years of operation
- 29 repository system and microservice functions checked on a daily basis

REPOSITORY TEAM

- Part of the UC Curation Center
- One Product/Service Manager
- One Technical Lead
- Two Sr. Developers
- One shared DevOps engineer

CLOUD VENDORS

- San Diego Supercomputer Center (academic)
- Amazon Web Services
- Wasabi Hot Cloud Storage



REPOSITORY AUTOSCALING THROUGH USE OF CONTAINERS AND OPERATIONAL METRICS

At California Digital Library (CDL), the digital preservation repository team is amid a paradigm shift that entails movement of the system's microservices and queueing capabilities to a fully containerized state. Meaning, each key service, such as Ingest, Inventory, and Storage will operate within independent yet orchestrated containers. There are numerous advantages to running a repository using this strategy.

This poster illustrates the repository's transition from its current cloud infrastructure configuration, where services run on one or more dedicated cloud compute hosts, to running services in containers. It captures elements such as infrastructure as code, orchestration, and vitally, using key metrics to determine when the system should scale its resources to handle variable deposit rates – all with the intention of optimizing throughput, costs and environmental impact.



PRE-MIGRATION STATE

Historically, CDL's digital preservation repository implemented a series of microservices, each of which drives a particular operation necessary for submitting, processing and storing digital materials. These microservices are based on the Open Archival Information System (OAIS) reference mode and include: the submission User Interface, Ingest, Storage, Inventory, Replication and Audit (fixity check). Underlying these operations is a comprehensive queueing system.

Since its inception, the repository's microservices have been running on AWS cloud infrastructure, primarily through use of multiple Elastic Compute Cloud (EC2) hosts, MySQL databases, temporary staging storage and finally S3 or S3-compliant cloud storage hosted by multiple service providers. Over time, the footprint of the repository has grown significantly. Not only has the amount of content being managed and preserved increased in size (currently 1.8PB), but the maximum rate

CONTAINERS AND ORCHESTRATION IN THE CLOUD

What's in a container? In essence, a container is a fully functional and portable cloud computing environment that surrounds an application or service. Inside a container, a service has all the resources it needs to run – including an allocation of memory, configuration files and any required software libraries and dependencies. Critically, containers can be quickly spun up, operated and then spun down when no longer needed. Managing containers in this manner is part of the orchestration process and is commonly called autoscaling. Autoscaling is one of the key reasons our team decided to migrate the repository's services to containers. Once fully migrated, the repository will no longer need to constantly run at what is gauged as maximum service capacity, regardless of the rate of deposit.

With orchestrated containers, the system will automatically add new service container instances only when they are needed. Conversely, when the rate of deposit slows, containers will be spun down and stopped.

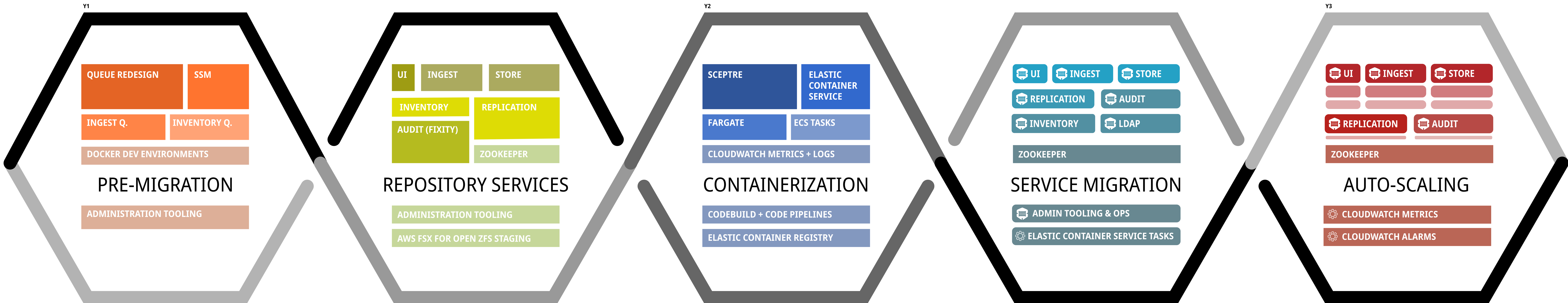
of deposit has grown in parallel. Individual submissions range from kilobytes per file to thousands of gigabytes that comprise hundreds of complex digital objects. This increased rate has necessitated constant use of multiple hosts per microservice. Given the high-availability nature of its services, the overall system requires numerous hosts to operate. Needless to say, the costs associated with such a system continue to grow, in both monetary and environmental forms. Such costs are what led us to migrate the repository's services to cloud-based containers.

Key operational metrics trigger the growth or reduction of the repository's footprint. Metrics such as total estimated bytes to process, number of queued submissions, the number of objects being sent to cloud storage and the oldest known unaudited item, among others.

ENVIRONMENTAL FOOTPRINT REDUCTION

Both large and small, incremental improvements to how the repository makes use of cloud compute resources have gained ground during the migration. For instance, hardware used to run ECS containers leverages AWS Graviton energy-saving processors which use up to 60% less energy than standard x86 processors (though we had already switched to Graviton-powered EC2s).

Using ECS with Fargate shifts infrastructure scaling entirely to AWS, which optimizes underlying server populations at cloud scale. Due to Fargate's serverless design, it abstracts infrastructure management and is generally able to promote higher multi-tenant host utilization across physical hosts, thus lessening un- or under-utilized capacity in the cloud. Sharing underlying hardware lowers the embodied carbon and electricity waste per application. Containers also make it easier to precisely allocate small amounts of CPU and memory, avoiding potential, heavy over-provisioning that can occur with individual EC2 instances. The team reviews compute metrics on a monthly basis to determine if further configuration-related optimizations can take place. It also purges unused images in ECR to minimize storage needs.



In advance of migrating to containers, each repository service needed to be stateless. That is to say, shutting it down would do no harm to its ability to process content. To achieve a set of stateless microservices, several building blocks were implemented.

Ingest Queue Redesign

The Merritt repository uses Apache ZooKeeper to manage queue events, such as estimating the size of incoming files and determining when to download them. The number of queue states was increased so the ingest process could be paused at any time, as could the Inventory service that writes metadata to the system's central MySQL database hosted in the cloud via AWS RDS.

AWS Systems Manager (SSM) Parameter Store

All configuration-related data and secrets were moved to the cloud-based SSM Parameter Store. Parameters can be leveraged on service startup and in some cases, dynamically.

Docker Development Environments

Early on the team set up EC2 instances that served as hosts on which to run Docker containers. These served as dev and test beds for future implementations.

Administration Tooling

In order to gain visibility into queueing, in-process batches, service versions, deployments, a variety of metrics and monitoring, and granular control over operations, extensive administrative tooling was put in place.

At this stage, all microservices were still running on AWS Elastic Compute Cloud (EC2) instances whose resources were tailored to maximum throughput.

Service Updates

As the Ingest queue redesign came online, the Ingest, Store and Inventory microservices were updated to take advantage of new queue states. The Administration tooling layer for the repository added re-queue capabilities, collection pausing and separately, cloud scanning.

Service Footprint

While still running on EC2s, microservice instances were manually scaled according to increasing rates of deposit. For example, the Audit service was scaled up to use four hosts as the amount of data the repository stewards grew past 1.2PB. This was done in order to maintain our advertised (SLA) 60-90 day fixity check cycle.

AWS FSx for Open ZFS

Also to accommodate increasing rates of deposit and larger individual files and batches, the team moved from using the Amazon Elastic File System (EFS) to AWS FSx for Open ZFS for staging content in advance of uploading it to cloud storage. The ZFS volume is 5TB in size and must be monitored to avoid reaching its maximum capacity inadvertently. That said, its advantages in terms of raw data throughput became obvious, as the overall system was able to move nearly twice as much content in a day as it once did with EFS (albeit, together with upgraded EC2s). Later, the team learns it must re-adopt EFS for use with ECS.

At CDL and within UC3, multiple teams begin containerizing their services and shared knowledge builds with respect to prototyping and implementation.

Sceptre and CloudFormation

The open source Sceptre codebase is adopted to streamline provisioning cloud-based resources, a.k.a. infrastructure as code. The Merritt team builds its own Sceptre GitHub repository to create AWS CloudFormation templates and configuration for automatically spinning up resources and containers.

AWS Elastic Container Service and Fargate

The AWS Elastic Container Service (ECS) is used to deploy orchestrated microservice-specific containers. Fargate manages assigning serverless compute to operational containers and the team learns about how to effectively assign specific numbers of virtual CPUs and memory. As memory configuration edges up, so does network bandwidth.

CloudWatch Metrics and Logs

Amazon CloudWatch is used to monitor services. Logs are routed to CloudWatch for review on a regular basis. Critically, the team learns how to implement CloudWatch Metrics and Alarms that will be used to trigger system scaling events in the future.

AWS CodeBuild, CodePipelines and Container Registry

CodeBuild and CodePipelines are adopted to build a CI/CD pipeline. Our GitHub repositories are connected to CodeBuild. CodePipelines are established for image generation purposes. Images and their containers are stored in the AWS Elastic Container Registry (ECR). The overall pipeline can be triggered on-demand and via check-ins. A tagging strategy is devised and tagged ECR images are moved into production when ECS services are redeployed.



Service migration to containers in the Merritt Stage and Production environment begins.

Merritt Admin, User Interface

The Merritt Admin, or administration layer of the repository, is the first service to migrate. A lambda-based web application built with the Sinatra framework, the Admin can interact with ZooKeeper, Inventory, LDAP (for user accounts), GitHub, CodeArtifact and ECR. It is eventually scaled to enable push-button, weekly stack redeployment and automated testing.

Merritt's UI is the first user-facing service to migrate. The team begins retiring EC2 UI hosts and reduces the number of running instances from three EC2 hosts to two ECS-managed containers.

The Audit service is migrated to containers. We discover that three containers can perform nearly the same amount of work that four custom-configured EC2 hosts could, at a lower overall cost. A fourth container is added to further reduce fixity check cycle time.

The Replication and Inventory services are migrated. LDAP is migrated as well and poses a unique challenge – data must be persisted and always available during deployments so users are not interrupted (EFS is employed for persistence).

Amid the migration, the team devises new methods to diagnose issues, and observe logs and service metrics. "Ops" and Ops-write" containers are launched, in part for these purposes.

ECS Tasks

Each containerized service is essentially an ECS Task. However the team begins employing ECS Tasks as a replacement for cron-driven events. Tasks now run service consistency checks, automated tests and drive automated DAMS-to-repository harvesting.



To date, all repository services have been containerized, with the exception of ZooKeeper. Our initial deployments to production of the Ingest and Storage services are set to occur approximately two weeks after IPRES concludes.

Auto-scaling with CloudWatch Metrics and Alarms

Auto-scaling is achieved by a combination of configuration (yaml) and template files that specify parameters for scaling purposes. These templates are directly linked to specific repository metrics that are captured live during microservice operations. You can find the current list of metrics at the top left corner of the poster.

Scaling occurs when a metric reaches a predefined threshold. At that point a CloudWatch Alarm is triggered. The alarm is either in an "OK" or "Alarm" state. Depending on the metric data and aforementioned templates, the ECS Task that is running a service handles increasing or decreasing the number of service containers in operation. As the team progresses, it continues to tweak Alarm settings and container resources to responsibly increase repository performance according to the live rate of deposit.

ZooKeeper (ZK) Migration

Merritt's implementation of Apache ZK will be the final service to migrate. It may be the most complex to migrate due to its need to persist queue data and maintain a quorum of containers. We hope that lessons learned during the migration of LDAP will apply.

Weekly Deployments

In the past Merritt hosts were patched on a monthly basis. They are now immutable (purged/recreated weekly) and automatically gain all available security patches offered through AWS and community maintained base images that run in our containers – further heightening the security bar achieved by the repository.

Block size in each section roughly indicates amount of effort, codebase complexity, service footprint and/or predominance. Implementations followed overarching themes of transparency and ease of service management.